

IDFC FIRST Bank Edge Delivery Services - Developer Guide v3

Date: 2026-04-24

Adobe

Table of Contents

IDFC FIRST Bank – Edge Delivery Services Developer Guide	3
1. Introduction	3
2. Quick Reference	3
3. Architecture Overview	4
4. xwalk Component Model System	6
5. Block Development for xwalk	8
6. Universal Editor Integration	10
7. Key Files Deep Dive	12
8. All 26 Blocks Reference	15
9. Design System	19
10. Templates	22
11. Sidekick Configuration	22
12. Local Development Setup	23
13. Git Workflow	24
14. Third-Party Dependencies	25
15. Troubleshooting	25
16. Resources	26

IDFC FIRST Bank – Edge Delivery Services Developer Guide

1. Introduction

This guide covers the IDFC FIRST Bank website built on **Adobe Experience Manager (AEM) Edge Delivery Services** using the **xwalk (crosswalk) authoring model** with the **Universal Editor**. This project uses AEM Author as the single content source. Authors create and edit pages directly in the AEM Author instance through the Universal Editor, and the Franklin delivery pipeline renders that content as optimized HTML at the edge.

The xwalk model provides a component-driven authoring experience while retaining the performance benefits of Edge Delivery Services: static markup, aggressive CDN caching, and Lighthouse scores near 100. Developers define block models as JSON, write lightweight JavaScript decorators and CSS, and rely on the platform's Eager-Lazy-Delayed (E-L-D) loading pattern to keep pages fast.

Key characteristics of this project:

- Content authored exclusively in AEM Author via Universal Editor
 - 26 custom blocks, each with its own JSON model definition
 - Modular JSON build system using `merge-json-cli`
 - Multi-section system for tabs and accordions built from grouped sections
 - Category navigation via fragment injection
 - Mayura template with blue/red sub-themes
 - SwiperJS for carousel/slider functionality
 - DOMPurify for HTML sanitization throughout the codebase
 - Comprehensive ESLint security rules (eslint-plugin-xwalk, eslint-plugin-secure-coding, eslint-plugin-browser-security)
-

2. Quick Reference

Environment URLs

Environment	URL
AEM Author	https://author-idfc-stage-65.adobecqms.net
Preview	https://main--idfc--aemsites.aem.page/
Live / CDN	https://main--idfc--aemsites.aem.live/
Local Dev	http://localhost:3000/

Key File Paths

File	Purpose
<code>scripts/scripts.js</code>	Main application script (~1833 lines)
<code>scripts/aem.js</code>	Core EDS library (loadBlock, decorateBlock, etc.)
<code>scripts/editor-support.js</code>	Universal Editor integration
<code>scripts/editor-support-rte.js</code>	Rich text editing support

component-definition.json	Built component definitions
component-models.json	Built component models
component-filters.json	Built component filters
paths.json	JCR-to-URL path mappings (14 entries)
head.html	HTML head configuration
helix-query.yaml	Query index configuration
helix-sitemap.yaml	Sitemap generation config
package.json	Dependencies and build scripts
.eslintrc.js	Linting and security rules
tools/sidekick/config.json	Sidekick plugin configuration

3. Architecture Overview

Technology Stack

The content delivery pipeline for this xwalk project follows this flow:

```

AEM Author Instance
  |
  v
Franklin Delivery Endpoint
(https://{author-host}/bin/franklin.delivery/{org}/{site}/{branch})
  |
  v
Edge Delivery CDN (aem.page / aem.live)
  |
  v
Browser (HTML + JS + CSS)

```

Content Source Configuration: In xwalk projects, the AEM Author instance serves as the content source, delivering markup via the Franklin delivery endpoint. The content source mapping is configured through the **Edge Delivery Config Service** – not through any repository file. The Config Service manages the connection between the AEM Author instance and the Edge Delivery pipeline, defining the markup endpoint URL pattern:

```
https://author-idfc-stage-65.adobecqms.net/bin/franklin.delivery/aemsites/idfc/main
```

This means content flows from AEM Author through the Franklin delivery layer, which converts JCR content into the flat HTML structure that Edge Delivery Services expects. The `paths.json` file then maps JCR content paths to public URL paths.

Eager-Lazy-Delayed (E-L-D) Loading Pattern

The `scripts.js` file orchestrates page loading through three phases:

loadEager – Runs first, handles everything needed for Largest Contentful Paint (LCP):

- Sets document language
- Decorates template and theme
- Loads theme spreadsheet configuration (design token overlay)
- Decorates the main element (buttons, icons, sections, auto-blocks)
- Preloads hero-heritage-cc LCP images
- Injects hidden H1 from metadata if configured
- Loads the first section and waits for the first image
- Loads fonts on desktop or if previously cached

loadLazy – Runs after LCP, loads remaining page infrastructure:

- Autolinks modals (click handler for /modals/ links) - Loads header

and footer - Loads the "Get App" banner fragment (mobile only) - Builds and injects breadcrumbs - Loads category navigation from fragment metadata - Loads all remaining sections - Rewrites relative links to production origin - Loads lazy styles and fonts - Initializes exit-intent modal

loadDelayed – Runs 3 seconds after page load: - Loads `delayed.js` which injects Google Tag Manager (GTM-M5CHMQ2Z)

Project Directory Structure

```

/
|-- blocks/                                # 26 block directories
|   |-- accordion/
|   |   |-- _accordion.json                # Block model/definition/filter
|   |   |-- accordion.js                  # Block decorator
|   |   |-- accordion.css                  # Block styles
|   |-- anchor-nav/
|   |-- banner/
|   |-- cards/
|   |-- cards-icon/
|   |-- cards-testimonial/
|   |-- category-nav/
|   |-- cc-hero-slider/
|   |-- columns/
|   |-- cta-sticky/
|   |-- embed/
|   |-- footer/                            # No _model.json (framework block)
|   |-- fragment/
|   |-- header/                            # No _model.json (framework block)
|   |-- hero/
|   |-- hero-heritage-cc/
|   |-- hotspot/
|   |-- library-metadata/                  # JSON only (no JS/CSS)
|   |-- link-to-upi/
|   |-- modal/                             # No _model.json (programmatic)
|   |-- overview/
|   |-- section-title/
|   |-- steps/
|   |-- table/
|   |-- tabs/
|   |-- tabs-upi-link/
|-- models/                                # Root-level model partials
|   |-- _component-definition.json          # Source definition with spread refs
|   |-- _component-models.json              # Source models with spread refs
|   |-- _component-filters.json             # Source filters with spread refs
|   |-- _page.json                          # Page metadata model
|   |-- _section.json                       # Section model/definition/filter
|   |-- _multi-section.json                 # Multi-section model
|   |-- _button.json                        # Button component
|   |-- _image.json                         # Image component
|   |-- _text.json                          # Text component
|   |-- _title.json                         # Title component
|   |-- partials/                           # Shared option sets
|       |-- _bg-fit.json
|       |-- _bg-position.json
|       |-- _sizes.json

```

```

|      |-- _widths.json
|
|-- component-definition.json      # BUILT output (do not edit)
|-- component-models.json         # BUILT output (do not edit)
|-- component-filters.json        # BUILT output (do not edit)
|
|-- scripts/
|   |-- scripts.js                # Main application (~1833 lines)
|   |-- aem.js                   # Core EDS utilities
|   |-- editor-support.js         # Universal Editor integration
|   |-- editor-support-rte.js     # RTE grouping for UE
|   |-- delayed.js               # GTM injection
|   |-- dompurify.min.js         # HTML sanitization library
|   |-- library-utils.js         # Sidekick library utilities
|   |-- sidekick-library.js      # Sidekick library support
|   |-- swiperjs/
|       |-- swiper-bundle.min.js
|       |-- swiper-bundle.min.css
|
|-- styles/
|   |-- styles.css               # Main styles + CSS custom properties
|   |-- fonts.css               # Inter font loading
|   |-- lazy-styles.css          # Deferred styles
|   |-- editor-styles.css        # Universal Editor-only styles
|
|-- templates/
|   |-- mayura/
|       |-- mayura.css           # Template-specific overrides
|
|-- tools/
|   |-- sidekick/
|       |-- config.json          # Sidekick configuration
|       |-- library.html         # Block library UI
|
|-- icons/                       # SVG icon library
|-- fonts/                       # Self-hosted Inter font files
|-- content/                     # Documentation and guides
|
|-- head.html                    # HTML head injections
|-- paths.json                  # JCR-to-URL mappings
|-- helix-query.yaml            # Query index config
|-- helix-sitemap.yaml          # Sitemap config
|-- package.json                # Dependencies and scripts
|-- .eslintrc.js                # ESLint configuration
|-- .github/
|   |-- workflows/
|       |-- main.yaml            # CI: lint on push
|       |-- cleanup-on-create.yaml # Initial repo cleanup

```

4. xwalk Component Model System

Three Core JSON Files

The Universal Editor uses three JSON files to understand how to present components to authors. These are the **built outputs** at the repository root:

component-definition.json – Defines what components exist and how they are created: - Groups components into categories: Default Content, Sections, Blocks - Each component specifies an `id`, `title`, and a `plugins.xwalk.page` object - The `page` object contains: `resourceType`, `template` (with `name`, `model`, `filter` references)

component-models.json – Defines the authoring fields for each component: - Each model has an `id` that matches the `model` reference in the definition - Contains an array of `fields`, each with `component type`, `name`, `label`, and optional properties - Field types include: `text`, `richtext`, `reference`, `aem-content`, `select`, `multiselect`, `boolean`, `number`, `date-time`, `tab`, `container`

component-filters.json – Controls which child components can be placed inside a container: - The `main` filter allows only `section` and `multi-section` - The `section` filter lists all 26+ allowed blocks - Block-level filters restrict child items (e.g., `accordion` only allows `accordion-item`)

How They Interrelate

component-definition.json		component-models.json		component-filters.json
"id": "accordion"		"id": "accordion"		"id": "accordion"
"model": "accordion"	----->	"fields": [...]		"components":
["accordion-item"]				
"filter": "accordion"	----->			

When an author adds an Accordion block: 1. The **definition** tells the Universal Editor what to create (`resourceType`, `template`) 2. The **model** defines what fields the author can edit (`style`, `CTA`, `open-item`, etc.) 3. The **filter** restricts what children can be added (only `accordion-item` components)

Modular Build System with merge-json-cli

The three root-level JSON files are **generated** – never edit them directly. The source files live in `models/` and individual `blocks/*/_blockname.json` files. The build uses `merge-json-cli` to combine them.

Source file structure: Each `_blockname.json` file contains up to three top-level arrays:

```
{
  "definitions": [ ... ],
  "models": [ ... ],
  "filters": [ ... ]
}
```

Spread reference syntax (`"..."`) allows referencing external JSON files:

```
{
  "groups": [
    {
      "title": "Blocks",
      "id": "blocks",
      "components": [
        { "...": "../blocks/*/_*_json#/definitions" }
      ]
    }
  ]
}
```

This glob pattern (`../blocks/*/_*_json#/definitions`) collects all `definitions` arrays from every block's JSON file and merges them into the output.

Partials in models/partial/ provide reusable option sets: - `_sizes.json` - spacing/gap size options - `_widths.json` - grid column width options - `_bg-fit.json` - object-fit options (cover, contain, etc.) - `_bg-position.json` - object-position options (center, top, etc.)

Referenced via spread syntax within field options:

```
{
  "component": "select",
  "name": "spacing",
  "options": [{ "...": "./partials/_sizes.json#/options" }]
}
```

Build Scripts

```
npm run build:json           # Build all three JSON files
npm run build:json:models    # merge-json-cli -i "models/_component-models.json" -o
                             "component-models.json"
npm run build:json:definitions # merge-json-cli -i "models/_component-definition.json" -o
                             "component-definition.json"
npm run build:json:filters    # merge-json-cli -i "models/_component-filters.json" -o
                             "component-filters.json"
```

Always run `npm run build:json` after modifying any model/definition/filter JSON file.

5. Block Development for xwalk

Block File Structure

Each block lives in `blocks/{blockname}/` and typically contains:

File	Purpose
<code>_blockname.json</code>	Component definition, model fields, and filter rules
<code>blockname.js</code>	JavaScript decorator function
<code>blockname.css</code>	Block-specific styles

Some blocks have additional assets (e.g., `cards/` includes `.avif` mask images, `tabs-upi-link/` includes an `.mp4`).

Exceptions: header and footer have no `_model.json` (they are framework blocks loaded programmatically). `modal` is loaded programmatically and has no model. `library-metadata` has only a JSON file (no JS or CSS).

Definition Structure

Each block definition uses the xwalk plugin to tell the Universal Editor how to instantiate the block:

```
{
  "title": "Hero",
  "id": "hero",
  "plugins": {
    "xwalk": {
      "page": {
        "resourceType": "core/franklin/components/block/v1/block",
        "template": {
          "name": "Hero",

```



```

        "model": "hero"
      }
    }
  }
}

```

- **resourceType:** Usually `core/franklin/components/block/v1/block` for blocks, `core/franklin/components/block/v1/block/item` for child items, and `core/franklin/components/section/v1/section` for sections
- **template.name:** The block name as it appears in the content markup
- **template.model:** References the model ID in `component-models.json`
- **template.filter:** (optional) References the filter ID for blocks with children

For blocks with child items (accordion, cards, tabs, etc.), the definition includes a default item template:

```

{
  "template": {
    "name": "Accordion",
    "model": "accordion",
    "filter": "accordion",
    "item1": {
      "sling:resourceType": "blocks/accordion/accordion-item",
      "name": "Accordion Item",
      "model": "accordion-item",
      "col_head": "Lorem Ipsum",
      "col_body": "<p>Lorem ipsum...</p>"
    }
  }
}

```

Model Field Types

Component Type	Description	Example Use
text	Single-line text input	Alt text, URLs, labels
richtext	Rich text editor (HTML output)	Block body content
reference	DAM asset picker	Images, backgrounds
aem-content	AEM content path picker	Fragment references, modal content
select	Single-select dropdown	Style variants
multiselect	Multi-select checkboxes	Multiple style classes
boolean	Toggle switch	Enable/disable features
number	Numeric input	Coordinates, indices
date-time	Date/time picker	Date fields
tab	Tab separator in properties panel	Organize fields into tabs
container	Collapsible field group	Group related optional fields

Field properties: - `name` – The JCR property name (also used as the CSS class or data attribute name) - `label` – Display label in the properties panel - `description` – Help text shown to authors - `value` – Default value - `valueType` – Data type (string, number, boolean) - `multi` – Allow multiple values (for reference) - `condition`

- Conditional visibility based on other field values - validation - Validation rules (rootPath, numberMin, customErrorMsg) - hidden - Hide the field from the UI

The `classes` Convention

The field named `classes` has special significance. When a block model includes a `select` or `multiselect` field named `classes`, the selected values are added as CSS classes to the block element in the rendered markup. This is the primary mechanism for block variants:

```
{
  "component": "select",
  "name": "classes",
  "label": "Style",
  "options": [
    { "name": "Standard", "value": "" },
    { "name": "All About", "value": "all-about-card" },
    { "name": "Key Benefits", "value": "key-benefits" }
  ]
}
```

In the rendered HTML, the block gets the selected class: `<div class="cards all-about-card">`.

eslint-plugin-xwalk max-cells Limits

The `xwalk/max-cells` ESLint rule enforces a maximum number of fields (cells) per block model row. This prevents overly complex block tables and ensures authoring usability:

```
'xwalk/max-cells': ['error', {
  section: 30,
  'multi-section': 30,
  accordion: 5,
  cards: 8,
  card: 7,
  'category-nav-item': 10,
  'section-title': 5,
  steps: 5,
  table: 6,
}],
```

If you add fields to a model, ensure the total does not exceed the configured limit for that component.

6. Universal Editor Integration

head.html Configuration

The `head.html` file loads the core scripts and configures the Universal Editor preview environment:

```
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<script src="/scripts/aem.js" type="module"></script>
<script src="/scripts/scripts.js" type="module"></script>
<link rel="stylesheet" href="/styles/styles.css"/>
<link rel="stylesheet" href="/scripts/swiperjs/swiper-bundle.min.css"/>
<script src="/scripts/swiperjs/swiper-bundle.min.js"></script>
<meta name="urn:adobe:aue:config:preview" content="main--idfc-edge--aemsites.aem.page"/>
```

The `urn:adobe:aue:config:preview` meta tag tells the Universal Editor which preview domain to use for rendering.

editor-support.js

This script is loaded **only** in the Universal Editor context. It handles live preview updates when authors make changes, without requiring a full page reload.

isEditor flag: Sets `window.hlx.isEditor = true` immediately, which other scripts use to suppress behaviors like exit-intent modals during editing.

Editor-specific CSS: Loads `editor-styles.css` which provides UE-only layout overrides (e.g., disabling animations in hero-heritage-cc, showing structured editing views).

Six AUE Events Handled:

Event	Trigger
<code>aue:content-patch</code>	Field value changed
<code>aue:content-update</code>	Content updated
<code>aue:content-add</code>	New component added
<code>aue:content-move</code>	Component reordered
<code>aue:content-remove</code>	Component deleted
<code>aue:content-copy</code>	Component duplicated

Additionally, `aue:ui-select` is handled to activate the correct tab panel when an author selects an element inside a tabs block.

applyChanges() – Three Scopes

When any content event fires, `applyChanges()` determines the scope and applies the update accordingly:

1. **Main-level update** (`element.matches('main')`) – Full page re-decoration:
 - Creates a new main element from the update
 - Injects category nav from fragment
 - Calls `decorateMain()` and `loadSections()` on the new main
 - Replaces the old main element
2. **Block-level update** (element is inside a `.block[data-aue-resource]`) – Targeted block refresh:
 - Finds the updated block markup
 - Decorates buttons, icons, and the block
 - Loads the block
 - Replaces just the block element
3. **Section or default content update** – Section-level refresh:
 - For sections: re-decorates the parent, handles block loading
 - For default content: replaces the element in place

All updates go through **DOMPurify sanitization** before being inserted into the DOM.

DOMPurify Usage

DOMPurify is loaded from `scripts/dompurify.min.js` and used throughout:

- `applyChanges()` sanitizes all incoming content from the Universal

Editor - `loadFragment()` sanitizes fetched fragment HTML - The global `DOMPURIFY` config object (from `aem.js`) defines allowed HTML profiles

- `ensureDOMPurify()` in `scripts.js` provides a safe lazy-loading

mechanism

editor-support-rte.js

Handles rich text field grouping for the Universal Editor. When AEM delivers rich text content, each paragraph comes as a separate element with `data-richtext-prop` attributes. This script:

1. Scans for elements with `data-richtext-prop` attributes
2. Collects consecutive siblings with the same resource/prop
3. Wraps them in a single `<div>` with `data-aue-type="richtext"` for in-place editing
4. Handles orphan elements that are not consecutive siblings
5. Uses a `MutationObserver` to catch dynamically added richtext-instrumented elements

editor-styles.css

Loaded only in the Universal Editor, this CSS file: - Disables all animations in hero-heritage-cc (prevents disorienting motion during editing) - Converts absolutely positioned sections to normal flow - Shows labeled section previews (Header CTA, Intro, Banner) - Displays thumbnail previews of background and logo images - Resets gradient text effects for readability - Ensures `.cards.all-about-card` is always visible

moveInstrumentation()

Critical function for xwalk block development. When a block decorator restructures DOM (which is common), the `data-aue-*` attributes that the Universal Editor needs for in-place editing must be transferred to the new elements:

```
export function moveInstrumentation(from, to) {
  moveAttributes(
    from,
    to,
    [...from.attributes]
      .map(({ nodeName }) => nodeName)
      .filter((attr) => attr.startsWith('data-aue-') || attr.startsWith('data-richtext-')),
  );
}
```

Usage in block decorators: Whenever you create a new wrapper or restructure a block's DOM, call `moveInstrumentation(originalElement, newElement)` to preserve editing capability.

isEditor Detection

Use `window.hlx?.isEditor` to detect the Universal Editor context:

```
// Skip exit-intent modals in editor
if (window.hlx?.isEditor) return;

// Detect via data attribute on main
const isInUniversalEditor = !!document.querySelector('main[data-aue-resource]');
```

7. Key Files Deep Dive

scripts.js (~1833 lines)

The central application script contains these subsystems:

Link Processing: - `processLink()` / `processLinks()` - Opens PDF and `my.idfcfirst` links in new tabs - `makeProdUrl()` - Rewrites `ww2.idfcfirst.bank.in` links to production origin

Fragment System: - `loadFragment()` - Fetches `.plain.html`, sanitizes with `DOMPurify`, fixes media paths, marks category-nav blocks - `decorateFragment()` - Default export for the fragment block, replaces block content with fragment sections

Modal System: - `autolinkModals()` - Click handler for links containing `/modals/`, imports `modal.js` dynamically - `getModalOptionsFromParent()` - Extracts theme data attributes from parent block - `loadExitIntentModal()` - Configures exit-intent triggers (mouse leave, back button, fast scroll, idle timeout, tab focus)

Section System: - `decorateSections()` - Groups children into `.default-content` and `.block-content` wrappers - `handleSectionMetadata()` - Processes section metadata: style, `backgroundColor`, grid, gap, spacing, height, background images, decoration images, block-content backgrounds - `setColorScheme()` - WCAG luminance-based light/dark scheme detection - `handleBackgroundImages()` - Creates responsive `<picture>` elements for section backgrounds

Multi-Section System: - `buildMultiSection()` - Finds sections with `data-is-multisection="true"`, collects grouped sections, creates tabs or accordion blocks from them - `appendMultiSectionBlock()` - Builds the block element and inserts into `.block-content`

Category Navigation: - `loadCategoryNavFragment()` - Reads `category-nav` page metadata, loads fragment, injects sections into main

- `loadCategoryNav()` - Creates wrapper in header, loads CSS, loads all

`category-nav` blocks, cleans up fragment sections

Breadcrumbs: - `buildBreadcrumbs()` - Builds breadcrumb trail from navigation tree or URL path - `generateBreadcrumbSchema()` - Creates `BreadcrumbList` JSON-LD schema - `injectBreadcrumbSchema()` - Injects schema into document head - Override via `breadcrumbsTitle` page metadata

Design Token Overlay: - `loadThemeSpreadSheetConfig()` - Reads design metadata, fetches design spreadsheet JSON, creates CSS custom property overrides via `<style>` element

Template Loading: - `loadTemplate()` - Loads template CSS and JS module (e.g., `templates/mayura/mayura.js`)

Hero Optimization: - `preloadHeroHeritageCcLcpImage()` - Preloads LCP image from `hero-heritage-cc` intro section - `prepareHeroForCLS()` - Restructures hero block DOM on desktop to prevent CLS

Security: - `MAX_ITERATION_LIMIT = 500` - Caps collection iteration to prevent DoS (CWE-400) - `ensureDOMPurify()` - Lazy-loads `DOMPurify` for HTML sanitization - All `innerHTML` assignments go through `DOMPurify`

aem.js

The core EDS library providing: - `loadBlock()`, `decorateBlock()`, `decorateBlocks()` - Block lifecycle - `loadSection()`, `loadSections()` - Section loading - `decorateButtons()`, `decorateIcons()` - Content decoration - `decorateTemplateAndTheme()` - Body class management - `loadCSS()`, `loadScript()` - Dynamic resource loading - `getMetadata()` - Page metadata reading - `buildBlock()` - Programmatic block creation - `waitForFirstImage()` - LCP optimization - `DOMPURIFY` - Sanitization configuration - `pickPicturePreloadUrl()` - Picture element preload URL selection - `toCamelCase()` - String utility

paths.json (14 Mappings)

Maps JCR content paths to public URL paths:

```

{
  "mappings": [
    "/content/idfc-edge:/",
    "/content/idfc-edge/config:/config.json",
    "/content/idfc-edge/headers:/helix/headers.json",
    "/content/idfc-edge/configuration:/helix/config.json",
    "/content/idfc-edge/redirects:/redirects.json",
    "/content/idfc-edge/metadata:/metadata.json",
    "/content/idfc-edge/designs/design:/design.json",
    "/content/idfc-edge/credit-card/metal-credit-card/mayura/design:/credit-card/metal-credit-card/mayura/design.json",
    "/content/idfc-edge/experiment-metadata:/experiment-metadata.json",
    "/content/idfc-edge/api/test:/api/test.json",
    "/content/idfc-edge/dept-best-practice/form/contact-form:/dept-best-practice/form/contact-form.json",
    "/content/dam/idfc-edge/icons:/icons/"
  ],
  "includes": [
    "/content/idfc-edge/",
    "/content/dam/idfc-edge/icons/",
    "/content/dam/idfc-edge/videos/"
  ]
}

```

The `includes` array defines which JCR paths are included in the delivery scope. The `mappings` array transforms JCR paths to clean URL paths.

.eslintrc.js (Security Rules)

Extends three security-focused plugin configurations:

- **eslint-plugin-xwalk** (`plugin:xwalk/recommended`) – Validates component model definitions, enforces `max-cells` limits
- **eslint-plugin-secure-coding** – Detects hardcoded credentials, ReDoS patterns, unsafe deserialization, format string injection, unchecked loop conditions, XPath/GraphQL/XXE injection, sensitive data exposure
- **eslint-plugin-browser-security** – Prevents `innerHTML` without sanitizers, `eval`, `postMessage` without origin check, sensitive data in `localStorage/sessionStorage/cookies`, insecure `WebSocket`, mixed content, clickjacking
- **eslint-plugin-sonarjs** (`plugin:sonarjs/recommended-legacy`) – Code quality and complexity rules

helix-query.yaml

Defines the query index used for search and content listings:

```

version: 1
indices:
  pages:
    include:
      - '/**'
    exclude:
      - '/*.json'
    target: /query-index.json
    properties:
      lastModified:
        select: none
        value: parseTimestamp(headers["last-modified"], "ddd, DD MMM YYYY hh:mm:ss GMT")
    robots:

```

```
select: head > meta[name="robots"]
value: attribute(el, "content")
```

helix-sitemap.yaml

Configures automatic sitemap generation:

```
sitemaps:
  default:
    source: /query-index.json
    destination: /sitemap.xml
    lastmod: YYYY-MM-DD
```

8. All 26 Blocks Reference

accordion

Purpose: Expandable FAQ/content sections with show/hide animation.

JS approach: Decorates each accordion-item with header/body structure, adds click handlers to toggle aria-expanded, generates schema.org FAQ microdata unless no-schema is set. Supports a CTA link inserted at a configurable position, and an optional "show more/less" toggle (when CTA URL is #).

CSS patterns: Smooth height transitions, arrow rotation on expand, dark-scheme/light-scheme variants.

Model fields (accordion): classes (multiselect: dark-scheme, light-scheme), no-schema (boolean), cta_location (number), link (text), linkText (text), open-item (number).

Model fields (accordion-item): col_head (richtext), col1_body (richtext).

Variants: dark-scheme, light-scheme.

anchor-nav

Purpose: In-page anchor navigation with smooth scrolling.

JS approach: Reads anchor-nav-link items, creates a horizontal navigation bar, highlights active section on scroll via IntersectionObserver.

CSS patterns: Sticky positioning, horizontal scroll on mobile, active indicator animation.

Model fields (anchor-nav-link): Link label and target section ID.

banner

Purpose: Promotional banner with rich content and image.

JS approach: Structures banner-item children into heading, content, and image areas.

CSS patterns: Full-width layout, responsive image positioning, gradient text headings in Mayura template.

Model fields (banner-item): Rich text content and image references.

cards

Purpose: Flexible card grid with SwiperJS carousel support.

JS approach: Decorates card items with image, tag, text, and optional modal/link. When `swipable` is enabled, initializes SwiperJS with configurable starting card and autoplay. Supports inline modal content or link to `/modals/` paths. Custom scrollbar handle in Mayura template.

CSS patterns: Grid layout, responsive columns, card hover effects, mask images (`rectangle-card-mask.avif`, `shadow-card-mask.avif`).

Model fields (cards): `classes` (select: `standard`, `all-about-card`, `key-benefits`, `experience-life`, `reward-points`, `blog-posts`, `earn-rewards`, `joining-perks`, `explore-other-cards`), `modalTheme` (select), `modalDialogBackgroundImageTexture` (reference), `modalPageBackgroundImage` (reference), `modalPageDecorationImage` (reference), `swipable` (boolean), `autoplayEnabled` (boolean), `startingCard` (number with min:0 validation).

Model fields (card): `image` (reference), `imageAlt` (text), `dividerImage` (reference), `cardTag` (text), `backgroundImageTexture` (reference), `text` (richtext), `cardLink` (aem-content), `cardLinkText` (text, hidden), `modalContent` (richtext).

Variants: `all-about-card`, `key-benefits`, `experience-life`, `reward-points`, `blog-posts`, `earn-rewards`, `joining-perks`, `explore-other-cards`.

cards-icon

Purpose: Icon-centric cards for feature highlights.

JS approach: Similar to cards but focuses on icon-first layout with compact content.

CSS patterns: Centered icon, smaller card sizing, grid layout.

cards-testimonial

Purpose: Customer testimonial cards with author attribution.

JS approach: Structures testimonial content with quote, author name, and designation.

CSS patterns: Quote styling, author attribution layout.

category-nav

Purpose: Multi-category navigation bar injected into the header.

JS approach: Complex system that collects all category-nav blocks on a page, builds a unified navigation bar, and moves it into the header's `.nav-wrapper`. Uses `resetUnifiedNavFlag()` to control rebuilding. Loaded from fragments referenced by page metadata `category-nav` field.

CSS patterns: Horizontal scrollable nav, sticky positioning below header, active category indicator.

cc-hero-slider

Purpose: Credit card hero section with animated slider.

JS approach: SwiperJS-powered slider for credit card hero content with multiple slides.

CSS patterns: Full-viewport hero, slide transitions, content overlay.

columns

Purpose: Multi-column content layout.

JS approach: Wraps column content and distributes into a CSS grid.

CSS patterns: Responsive grid columns, stacks on mobile.

cta-sticky

Purpose: Sticky call-to-action bar that remains visible while scrolling.

JS approach: Creates a fixed-position CTA bar, manages show/hide based on scroll position.

CSS patterns: Fixed positioning, gradient background (Mayura pewter), z-index management.

embed

Purpose: Embeds external media (YouTube, Vimeo, Twitter).

JS approach: Auto-detected from links matching embed platform patterns (youtu.be, vimeo, twitter.com). Created programmatically by `buildEmbedBlocks()` in `scripts.js`.

CSS patterns: Responsive iframe container with 16:9 aspect ratio.

footer

Purpose: Global site footer.

JS approach: Framework block loaded by `loadFooter()` from `aem.js`. No model definition – content comes from the footer content fragment.

CSS patterns: Multi-column layout, link lists, copyright section.

fragment

Purpose: Content fragment inclusion.

JS approach: Fetches `.plain.html` for the referenced path, sanitizes with `DOMPurify`, decorates the content, and replaces the block. Also auto-loaded for links containing `/fragments/`.

CSS patterns: Inherits parent section styling.

Model fields: Fragment path (`aem-content` or `text`).

header

Purpose: Global site header with navigation.

JS approach: Framework block loaded by `loadHeader()` from `aem.js`. Includes the "Get App" banner on mobile and the category-nav wrapper.

CSS patterns: Fixed positioning, hamburger menu, nav-sections with mega-menu.

hero

Purpose: Standard hero banner with image and text.

JS approach: Separates image and text content, optimizes image for LCP. `prepareHeroForCLS()` restructures DOM on desktop to prevent Cumulative Layout Shift.

CSS patterns: Full-width background image, text overlay, responsive layout.

Model fields: `image` (reference), `imageAlt` (text), `text` (richtext).

hero-heritage-cc

Purpose: Premium credit card hero with multi-phase animation sequence.

JS approach: Complex animated block with three sections: header CTA, intro (with logo reveal animation), and banner (main content). Preloads LCP image from intro section. Shows gradient color preview and structured editing layout in Universal Editor via `editor-styles.css`.

CSS patterns: Absolute positioning for animation layers, clip-path for logo reveals, gradient overlays, animation keyframes. All animations disabled in Universal Editor.

Model fields: Multiple sections with images, logos (Hindi/English), gradient colors, CTA buttons, and banner content.

hotspot

Purpose: Interactive image with clickable hotspot markers.

JS approach: Positions hotspot-item markers at X/Y coordinates over a base image, shows tooltip on click/hover.

CSS patterns: Relative positioned container, absolute positioned markers, tooltip popover.

Model fields (hotspot): image (reference), id (text), hotspot-text (richtext).

Model fields (hotspot-item): tooltip-text (richtext), x-hotspot (number), y-hotspot (number).

library-metadata

Purpose: Metadata block for the Sidekick block library.

JS approach: No JavaScript – this is a metadata-only block used by the Sidekick library plugin to categorize and describe blocks.

link-to-upi

Purpose: Deep link to UPI payment apps.

JS approach: Constructs UPI deep links for mobile payment apps.

CSS patterns: Button-style link with app icons.

modal

Purpose: Dialog/popup overlay system.

JS approach: Programmatic block (no model definition). `openModal()` fetches content from a path, creates a `<dialog>` element, supports themes (modal-mayura-blue, modal-mayura-silver) with texture backgrounds and decoration images. Used by cards (inline modal content), exit-intent system, and autolink modals.

CSS patterns: Backdrop gradient, dialog positioning, theme variants, close button.

overview

Purpose: Product overview section with structured content.

JS approach: Structures overview content into labeled sections with images and text.

CSS patterns: Side-by-side or stacked layout.

section-title

Purpose: Styled section heading.

JS approach: Wraps title content with decorative elements.

CSS patterns: Gradient text, centered alignment.

steps

Purpose: Step-by-step process visualization.

JS approach: Numbers and connects step items with visual connectors.

CSS patterns: Numbered circles, connecting lines, responsive stacking.

table

Purpose: Responsive data tables with background support.

JS approach: Creates proper `<table>` markup from block content, supports background colors/images per table via section metadata.

CSS patterns: Responsive table with horizontal scroll on mobile, striped rows.

tabs

Purpose: Tabbed content panels.

JS approach: Creates tab buttons from item labels, shows/hides panels on click, supports keyboard navigation. Also used as the output of multi-section grouping.

CSS patterns: Tab bar with active indicator, panel content area. Three styles: standard, silver-blue-text, blue-white-text.

Model fields (tabs): `classes` (select: Standard, Silver with dark text, Dark with white text).

Model fields (tabs-item): `tabName` (text), `content` (richtext).

tabs-upi-link

Purpose: Tabbed UPI link section with phone animation.

JS approach: Similar to tabs but includes a phone animation video (phone-animation-516px.mp4) and UPI deep link functionality.

CSS patterns: Tab layout with embedded video, UPI link buttons.

9. Design System

CSS Custom Properties

The design system is defined in `styles/styles.css` through CSS custom properties on `:root`.

Color Tokens

```
/* Reds */
--color-red-300: #d13d3f;
--color-red-400: #9c1d26;
--color-red-500: #902a2b;
--color-red-600: #801c1e;
--color-red-800: #6e0a0f;
--color-red-900: #480a14;

/* Blues */
--color-blue-200: #007bff;
--color-blue-700: #2E3E79;
--color-blue-800: #2d3679;
--color-blue-900: #182659;

/* Purples */
--color-purple-400: #563b80;
--color-purple-800: #293871;
```

```

--color-purple-900: #08133c;

/* Neutrals */
--color-white: #fff;
--color-white-100: #f5f5f5;
--color-white-200: #eee;
--color-gray-100: #DDDBDA;
--color-gray-200: #dbdbde;
--color-gray-300: #6c757d;
--color-gray-400: #505050;
--color-gray-500: #4b4b4b;
--color-gray-700: #25243B;
--color-gray-900: #222;
--color-pewter: #7A766D;
--color-black: #000;

```

Gradient Tokens

The project uses a rich set of gradients for premium visual effects:

```

--gradient-red-purple: linear-gradient(165deg, #D94751 0%, #8C2034 40%, #732C53 80%, #513873 100%);
--gradient-blue: linear-gradient(137deg, #08133C 4.07%, #2E3E79 75.46%);
--gradient-indigo: linear-gradient(308deg, #110140 4.16%, #1E2A8E 44.91%, #010740 88.8%);
--gradient-pewter: linear-gradient(270deg, #332B28 0%, ... var(--color-pewter) 90.18%, #433C32 100%);
--gradient-silver: linear-gradient(208deg, #C8C6C6 0%, #F1F0F0 53.5%, #C8C6C6 100%);
--gradient-metal-red: linear-gradient(90deg, #40010A 0%, #840114 100%);
--gradient-metal-blue: linear-gradient(308deg, #110140 4.16%, #1E2A8E 44.91%, #010740 88.8%);
--gradient-metal-silver: linear-gradient(138deg, #999 -15.44%, ... #EFEFEF 87.3%);

```

Additional gradients: --gradient-translucent, --gradient-radial-translucent, --gradient-pearl, --gradient-red-black, --gradient-pink-red, --gradient-earn-rewards, --gradient-red, --gradient-silver-red, --gradient-light-red, --gradient-black-90, --gradient-black, --gradient-conic-silver, --modal-backdrop-gradient.

Typography

```

--body-font-family: 'Inter', 'inter-fallback', sans-serif;
--heading-font-family: var(--body-font-family);

/* Body sizes */
--body-font-size-xl: 1.25rem; /* 20px */
--body-font-size-l: 1.125rem; /* 18px */
--body-font-size-m: 1rem; /* 16px */
--body-font-size-s: 0.875rem; /* 14px */
--body-font-size-xs: 0.75rem; /* 12px */
--body-font-size-xxs: 0.625rem; /* 10px */

/* Heading sizes */
--heading-font-size-xxl: 2.8125rem; /* 45px */
--heading-font-size-xl: 2.25rem; /* 36px */
--heading-font-size-l: 2rem; /* 32px */
--heading-font-size-m: 1.75rem; /* 28px */
--heading-font-size-s: 1.5rem; /* 24px */
--heading-font-size-xs: 1.375rem; /* 22px */

```

The Inter font uses a local Arial fallback with `size-adjust: 106.001%` to minimize CLS.

Spacing

```
--spacing-xxl: 72px;
--spacing-xl: 60px;
--spacing-l: 48px;
--spacing-m: 36px;
--spacing-s: 24px;
--spacing-xs: 12px;
--spacing-xxs: 8px;
```

Grid System

```
--grid-container-width: 1160px;
--grid-column-width: calc(var(--grid-container-width) / 12);
--grid-gutter-width: 16px;
```

Breakpoints

The primary breakpoint is **990px**:

```
const MEDIA_QUERIES = {
  mobile: window.matchMedia('(max-width: 599px)'),
  tablet: window.matchMedia('(min-width: 600px) and (max-width: 989px)'),
  desktop: window.matchMedia('(min-width: 990px)'),
};
```

These media queries are cached as `matchMedia` objects and reused throughout `scripts.js` for responsive behavior (section heights, background images, app banner visibility).

Button Tokens

```
--btn-radius: 2.4em;
--btn-primary-bg: var(--color-red-500);
--btn-primary-text: var(--color-white);
--btn-primary-border: var(--btn-primary-bg);
--btn-secondary-bg: var(--color-white);
--btn-secondary-border: var(--color-red-500);
--btn-secondary-text: var(--color-red-500);
--btn-tertiary-bg: var(--color-red-300);
--btn-tertiary-border: var(--color-red-300);
--btn-tertiary-text: var(--color-white);
```

Navigation Heights

```
--nav-height: 80px;
--category-nav-height: 46px;
--get-app-banner-height: 77px;
```

Design Token Overlay System

The project supports per-page design token overrides via spreadsheet data. The `loadThemeSpreadSheetConfig()` function in `scripts.js`:

1. Reads the `design` metadata field from the page

2. Fetches the corresponding JSON file (e.g., /design.json or /credit-card/metal-credit-card/mayura/design.json)
3. Iterates over rows with Property, Value, Section, and Block columns
4. Generates CSS rules that override custom properties at the appropriate specificity:
 - No Section/Block: :root { --Property: Value; }
 - Section only: main .section.{Section} { --Property: Value; }
 - Section + Block: main .section.{Section} .block.{Block} { --Property: Value; }
 - Special blocks: default, image, text, button, title map to specific selectors

10. Templates

Mayura Template

The Mayura template (templates/mayura/mayura.css) is designed for premium credit card product pages. It provides:

Base overrides: - Tertiary button: silver gradient with dark text (overrides red default) - Body background: conic silver gradient - Border radius: 20px

Blue sub-theme (.mayura.blue): - Heritage CC intro gradient: indigo - Benefit cards background: blue gradient - Tabs: silver background, blue-700 active - Modal: metal blue background - Hindi logo: pewter gradient text - Silver modal text: blue-900 with blue gradient clip

Red sub-theme (.mayura.red): - Heritage CC intro gradient: dark red - Benefit cards background: metal red gradient - Tabs: red-800 active - Modal: metal red background - Hindi logo: rose gradient text - Silver modal text: red-900 with metal red gradient clip

Shared Mayura styles: - Card/tab headings: metal red gradient text with clip - Dark scheme headings: metal silver gradient text - Banner headings: cream (#fbf8f2) gradient text - Custom Swiper scrollbar with drag handle image (scrollbar-handle.svg) - Accordion CTA: pewter gradient button - Sticky CTA: pewter gradient, no border - Column heading margins reset - Table CTA centering

The sub-theme is set via the page's theme metadata field, which adds the class to the <body> element (e.g., <body class="mayura blue">).

11. Sidekick Configuration

tools/sidekick/config.json

```
{
  "project": "IDFC",
  "editUrlLabel": "AEM Editor",
  "editUrlPattern": "{{contentSourceUrl}}{{pathname}}?cmd=open",
  "plugins": [
    {
      "id": "library",
      "title": "Library",
      "environments": ["preview", "live", "dev"],
      "url": "/tools/sidekick/library.html"
    }
  ]
}
```

editUrlPattern: The `?cmd=open` parameter opens the page in the Universal Editor when the author clicks "Edit" in the Sidekick.

Library plugin: Available in all environments, provides a block library UI at `/tools/sidekick/library.html` where authors can browse available blocks and see examples.

12. Local Development Setup

Prerequisites

- Node.js 20 or later
- npm
- AEM CLI (install globally via npm with the `aem-cli` package from the adobe scope)

Initial Setup

```
# Clone the repository
git clone https://github.com/aemsites/idfc.git
cd idfc

# Install dependencies
npm install

# Build the JSON model files (required before first run)
npm run build:json
```

Running the Dev Server

```
# Start the local AEM development server
aem up
```

This starts a local proxy at `http://localhost:3000/` that serves your local code files while fetching content from the Edge Delivery preview endpoint. Changes to JS, CSS, and JSON files are reflected immediately without restart.

Build Commands

```
npm run build:json      # Build all component JSON files
npm run lint            # Run both JS and CSS linting
npm run lint:js         # ESLint (JS + JSON files)
npm run lint:css        # Stylelint (block and style CSS)
npm run fix-js          # Auto-fix JS lint issues
npm run fix-css         # Auto-fix CSS lint issues
```

Development Workflow

1. **Create or modify a block:** Add/edit files in `blocks/{blockname}/`
2. **Update the model:** Edit `blocks/{blockname}/_blockname.json`
3. **Rebuild JSON:** Run `npm run build:json`
4. **Test locally:** `aem up` and navigate to a page that uses the block
5. **Test in Universal Editor:** Push to a branch and test on the preview URL
6. **Lint before committing:** `npm run lint`

Testing

- **Local preview:** `http://localhost:3000/{page-path}`
- **Preview environment:** `https://{branch}--idfc--aemsites.aem.page/{page-path}`
- **Live environment:** `https://{branch}--idfc--aemsites.aem.live/{page-path}`
- **Universal Editor:** Open the AEM Author page and click "Edit" in the Sidekick

13. Git Workflow

GitHub Actions

main.yaml – Runs on every push to any branch:

```
name: Build
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Use Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
```

This CI pipeline ensures all code passes ESLint (JS + JSON) and Stylelint (CSS) before merge.

cleanup-on-create.yaml – One-time workflow that runs on repository creation to remove template files and initialize README/PR template with actual repository values.

Husky Pre-Commit Hooks

The project uses Husky v9 for Git hooks:

```
.husky/
pre-commit      # Shell script hook
pre-commit.mjs  # Node.js hook logic
```

The pre-commit hook runs linting before each commit, preventing lint failures from reaching CI.

PR Requirements

1. All CI checks must pass (lint on push)
2. Code changes should include updated JSON builds if model files changed
3. Pre-commit hooks must pass locally
4. Branch naming should follow standard conventions (feature/, fix/, etc.)

14. Third-Party Dependencies

Build Tools

Package	Version	Purpose
merge-json-cli	^1.0.4	Merges modular JSON files with spread syntax into single output files
npm-run-all	^4.1.5	Run multiple npm scripts in parallel
husky	9.1.1	Git hooks for pre-commit linting

Linting

Package	Version	Purpose
eslint	8.57.1	JavaScript linting
eslint-config-airbnb-base	15.0.0	Base style rules
eslint-plugin-xwalk	v0.1.3 (GitHub)	xwalk model validation and max-cells enforcement
eslint-plugin-json	3.1.0	JSON file linting
eslint-plugin-import	2.32.0	Import/export rules
eslint-plugin-secure-coding	^3.1.3	Security-focused coding rules
eslint-plugin-browser-security	^1.2.3	Browser security best practices
eslint-plugin-sonarjs	^3.0.7	Code quality and complexity
stylelint	16.23.1	CSS linting
stylelint-config-standard	39.0.0	Standard CSS rules
babel-core + babel-eslint-parser (babel scoped packages)	^7.29.0 / 7.28.0	ES module parsing for ESLint

Runtime Libraries (Bundled)

Library	Location	Purpose
SwiperJS	scripts/swiperjs/	Touch-enabled carousel/slider (used by cards, cc-hero-slider, tabs-upi-link)
DOMPurify	scripts/dompurify.min.js	HTML sanitization for XSS prevention

Both SwiperJS and DOMPurify are loaded globally via `head.html` (SwiperJS) or on-demand (DOMPurify via `ensureDOMPurify()`).

15. Troubleshooting

Common xwalk Development Issues

Block not appearing in Universal Editor component list: - Verify the `_blockname.json` has a valid definitions array with the correct id - Run `npm run build:json` and check that the block appears in `component-definition.json` - Confirm the block ID is listed in the appropriate filter (section or multi-section) in `component-filters.json`

Model fields not showing in properties panel: - Check that the `model` reference in the definition matches a model id in `component-models.json` - Run `npm run build:json` to regenerate - Verify JSON syntax with `npm run lint:js`

Changes not reflecting in Universal Editor: - The editor caches aggressively. Try hard-refreshing the Universal Editor page - Verify your branch code is pushed and synced - Check the `urn:adobe:aue:config:preview` meta tag in `head.html` matches your branch

moveInstrumentation issues: - If a block restructures DOM but editing stops working, ensure `moveInstrumentation(from, to)` is called for every element that had `data-aue-*` attributes - Check that `data-richtext-*` attributes are also transferred

xwalk/max-cells lint error: - You have more fields in a model than the configured limit - Either reduce fields, group them in container components, or increase the limit in `.eslintrc.js`

Section metadata not applying: - Ensure the section-metadata block is a direct child of the section - Check the key names are lowercase in `handleSectionMetadata()` - Special keys (style, grid, gap, spacing, etc.) are handled separately from data attributes

Multi-section not building: - All sections in a group must have the same `data-multisection` value - The container section must have `data-is-multisection="true"` - The `category` field determines whether tabs or accordion is built

Category nav not loading: - Check that the page has a `category-nav` metadata field pointing to a valid fragment - The fragment must contain category-nav blocks - Framework pages (`/framework/` path) skip category nav loading

DOMPurify not loaded: - Call `ensureDOMPurify()` and await it before using `window.DOMPurify` - In `editor-support.js`, `DOMPurify` is loaded via `loadScript()` before sanitizing

Design token overlay not working: - Ensure the page has a `design` metadata field - The JSON file path must be mapped in `paths.json` - Check the spreadsheet has Property, Value, Section, Block columns

Local dev server not reflecting changes: - JSON changes require `npm run build:json` before they take effect - CSS and JS changes should be instant with `aem up` - Clear browser cache if styles seem stale

Lint failures in CI: - Run `npm run lint` locally before pushing - Security lint rules (`secure-coding`, `browser-security`) are strict - review the error messages for specific rule violations - Use `DOMPurify.sanitize()` instead of raw `innerHTML` assignments

16. Resources

Adobe Documentation

- [AEM Edge Delivery Services](#)
- [Universal Editor Documentation](#)
- [xwalk Component Model Definitions](#)
- [Block Collection](#)
- [Edge Delivery Config Service](#)
- [AEM CLI](#)

Project-Specific

- [GitHub Repository](#)
- [AEM Author Instance](#)
- [Preview Site](#)
- [Live Site](#)

Tools

- [merge-json-cli](#)

- [eslint-plugin-xwalk](#)
- [SwiperJS](#)
- [DOMPurify](#)
- [Stylelint](#)
- [Husky](#)